Kodbilaga (Krav 11): Effektiva ljuskällor, TFYY51, grupp GRYb1

Tagg v1.0.0, commit 1770adba3c0e81e85aaf82bb0f39354daa1f5a96
Gjord 2026-10-19 med git show v1.0.0:spectrometer/<fil>, filerna oförändrade. Kontroll: git rev-parse v1.0.0^{commit}
Fem avsnitt: spectrometer/calc.py, spectrometer/source.py, spectrometer/plot.py, spectrometer/files.py, spectrometer/main.py.
Raderna är numrerade som i filen; en rad som inte får plats fortsätter på nästa rad utan nummer.

1. spectrometer/calc.py (264 rader, blob 6b2f09d)

```
   1  """All räkning i programmet, bara med NumPy: spektrum, medel, våglängd, mörker, respons och transmission.
   2
   3  Används av main.py. Filen vet ingenting om state, filer eller kameran.
   4  """
   5  import numpy as np
   6  from numpy.polynomial import polynomial as poly
   7
   8  # Variables
   9  ## Channels
  10  CHANNEL_INDEX = {"blue": 0, "green": 1, "red": 2}
  11  CHANNELS = ("gray", "sum", "red", "green", "blue")
  12
  13  ## Grey weights
  14  blue_weight: float = 0.114
  15  green_weight: float = 0.587
  16  red_weight: float = 0.299
  17
  18
  19  def _check(name, a):
  20      """Kontrollera att a är en float64-array med bara ändliga tal.
  21
  22      Parametrar: name: argumentets namn som text, för felmeddelandet (str).
  23                  a: värdet som kontrolleras.
  24      Returnerar: None.
  25      Fel: ValueError med input.invalid_argument om a inte är en
  26           float64-array, eller om a innehåller inf eller NaN.
  27      """
  28
  29      if not isinstance(a, np.ndarray) or a.dtype != np.float64:
  30          raise ValueError(f"input.invalid_argument: {name} måste vara en float64-array")
  31
  32      if not np.all(np.isfinite(a)):
  33          raise ValueError(f"input.invalid_argument: {name} får bara innehålla ändliga tal")
  34
  35
  36  def prepare(image, roi, axis, channel):
  37      """Skär ut ROI:n som float64, väljer kanal och vänder vid vertical.
  38
  39      Parametrar: image: bilden, (H, W, 3) uint8 i BGR och counts, eller float64 efter mörkeravdrag;
  40      roi: (x, y, width, height) i pixlar; axis: "horizontal" eller "vertical"; channel: ett namn i CHANNELS.
  41      Returnerar: tabellen, (rader, kolumner) float64 i counts, med spektrumet längs kolumnerna.
  42      Fel: ValueError med input.invalid_argument om bilden, ROI:n, axis eller channel är fel.
  43      """
  44      if not isinstance(image, np.ndarray) or image.dtype not in (np.uint8, np.float64):
  45          raise ValueError("input.invalid_argument: image måste vara en uint8- eller float64-array")
  46      if image.ndim != 3 or image.shape[2] != 3:
  47          raise ValueError(f"input.invalid_argument: image måste ha formen (H, W, 3), fick {image.shape}")
  48      x, y, width, height = roi
  49      if width <= 0 or height <= 0:
  50          raise ValueError(f"input.invalid_argument: roi måste ha bredd och höjd större än 0, fick {roi}")
  51      h, w = image.shape[:2]
  52      if x < 0 or y < 0 or x + width > w or y + height > h:
  53          raise ValueError(f"input.invalid_argument: roi {roi} ligger utanför bilden ({w} bred, {h} hög)")
  54      if channel not in CHANNELS:
  55          raise ValueError(f"input.invalid_argument: okänd kanal {channel}, tillåtna är {CHANNELS}")
  56      if axis not in ("horizontal", "vertical"):
  57          raise ValueError(f"input.invalid_argument: okänd riktning {axis}, tillåtna är ('horizontal',
              'vertical')")
  58      view = image[y:y + height, x:x + width]
  59      cut = view.astype(np.float64)
  60      blue = cut[:, :, CHANNEL_INDEX["blue"]]
  61      green = cut[:, :, CHANNEL_INDEX["green"]]
  62      red = cut[:, :, CHANNEL_INDEX["red"]]
  63      if channel in CHANNEL_INDEX:
  64          result = cut[:, :, CHANNEL_INDEX[channel]]
  65      elif channel == "sum":
  66          result = blue + green + red
  67      elif channel == "gray":
  68          # Gråvärdet är varje färg gånger sin vikt, ihop (vikterna summerar till 1). Se rev7, RX01.
  69          result = red_weight * red + green_weight * green + blue_weight * blue
  70      if axis == "vertical":
  71          result = result.T
  72      return result
  73
  74
  75  def extract_spectrum(table):
```

```
    """
    Takes the average of the rows in the table, one amount per column.
    Parameters: Table: the table from prepare(), form (rows, columns), float64 (ndarray).
    Returns: Average per column, form (columns,), float64 (ndarray).
    Error: ValueError with input.invalid_argument if table is not float 64, is inf, is NaN, does not have two dimensions or is empty.
    """

    _check("table", table)

    # Kontrollera form
    if table.ndim != 2:
        raise ValueError(f"input.invalid_argument: table must have 2 dimensions, got shape {table.shape}")

    if table.size == 0:
        raise ValueError("input.invalid_argument: table is empty")

    # Spektrumet är medlet av raderna, ett värde per kolumn. Se rev7, RX05.
    return np.mean(table, axis=0)


def average_spectra(stack):
    """Ger medel och spridning av flera spektrum.

    Parametrar: stack: en float64-array med formen (N, P), en rad per bild.
    Returnerar: mean och stddev, båda (P,) float64; stddev är NaN när N är 1.
    Fel: ValueError med input.invalid_argument om stack inte är 2D, inte float64 eller är tom.
    """
    if stack.ndim != 2:
        raise ValueError(f"input.invalid_argument: stack måste vara 2D (en rad per bild), fick ndim {stack.ndim}")
    if stack.dtype != np.float64:
        raise ValueError("input.invalid_argument: stack måste vara en float64-array")
    if stack.size == 0:
        raise ValueError("input.invalid_argument: stack är tom")
    mean = np.mean(stack, axis=0)
    n = stack.shape[0]
    if n == 1:
        # En enda bild har ingen spridning: NaN utan varning. Se rev7, RX07.
        stddev = np.full(len(mean), np.nan)
    else:
        # Stickprovsstandardavvikelse, delad med N - 1 (ddof=1). Se rev7, RX06.
        stddev = np.std(stack, axis=0, ddof=1)
    return mean, stddev


def evaluate_wavelength(pixels, coefficients):
    """Ger våglängden i nm vid varje pixel.

    Parametrar: pixels: pixlar räknade i hela bilden, (P,) float64; coefficients: 2 eller 3 koefficienter, float64, a0 först (nm, nm per pixel, nm per pixel i kvadrat).
    Returnerar: våglängderna i nm, (P,) float64.
    Fel: ValueError med input.invalid_argument om en array inte är float64 med ändliga tal, eller om coefficients inte har 2 eller 3 värden.
    """
    _check("pixels", pixels)
    _check("coefficients", coefficients)
    if coefficients.size not in (2, 3):
        raise ValueError(f"input.invalid_argument: coefficients ska ha 2 eller 3 värden, fick {coefficients.size}")
    # Våglängd = a0 + a1·p (+ a2·p²), a0 först. Se rev7, RX09.
    return poly.polyval(pixels, coefficients)


def fit_wavelength(pixels, wavelengths, degree):
    """Anpassar ett polynom av grad 1 eller 2 till anpassningspunkterna.

    Parametrar: pixels: pixlar i hela bilden, (K,) float64; wavelengths: kända våglängder i nm, (K,) float64; degree: 1 eller 2.
    Returnerar: degree + 1 koefficienter, float64, a0 först.
    Fel: ValueError med input.invalid_argument om graden inte är 1 eller 2, arrayerna är fel, samma pixel står två gånger eller punkterna är för få.
    """
    if degree not in (1, 2):
        raise ValueError(f"input.invalid_argument: degree ska vara 1 eller 2, fick {degree}")
    _check("pixels", pixels)
    _check("wavelengths", wavelengths)
    if pixels.shape != wavelengths.shape:
        raise ValueError(
            f"input.invalid_argument: pixels och wavelengths har olika form, {pixels.shape} och {wavelengths.shape}"
        )
    unique = np.unique(pixels)
    if unique.size < pixels.size:
        raise ValueError("input.invalid_argument: samma pixel står två gånger")
```

```
157      if pixels.size < degree + 1:
158          raise ValueError(f"input.invalid_argument: grad {degree} behöver minst {degree + 1} punkter, fick
                 {pixels.size}")
159      # Minsta kvadrat-anpassning, koefficienterna i stigande ordning. Se rev7, RX10.
160      return poly.polyfit(pixels, wavelengths, degree)
161
162
163  def subtract_dark(image, dark):
164      """Drar av mörkret från bilden, utan att klippa negativa tal.
165
166      Parametrar: image och dark: (H, W, 3) float64 i counts.
167      Returnerar: image minus dark, en ny (H, W, 3) float64-array i counts.
168      Fel: ValueError med input.invalid_argument om en array inte är float64 med ändliga tal eller formerna
             skiljer sig.
169      """
170      _check("image", image)
171      _check("dark", dark)
172      if image.shape != dark.shape:
173          raise ValueError(f"input.invalid_argument: mörkret har formen {dark.shape} men bilden har
                 {image.shape}")
174      # Signal utan mörker = bild minus mörker, i float64 så att inget slår runt. Se rev7, RX03.
175      return image - dark
176
177
178  def build_response(wavelengths, measured, known_wavelengths, known_values):
179      """Räknar instrumentets relativa respons ur en lampmätning med känd fördelning.
180
181      measured får ha NaN där lampmätningen saknar värde, därför ingen _check på den.
182
183      Parametrar: wavelengths: lampmätningens våglängder i nm, (P,); measured: lampmätningen i counts, (P,), får
             ha NaN;
184      known_wavelengths: kända våglängder i stigande nm, (K,); known_values: känd relativ effekt, (K,).
185      Returnerar: responsen, (P,) float64, normerad så att största värdet är 1, NaN där den inte går att räkna.
186      Fel: ValueError med input.invalid_argument vid olika form eller färre än två kända punkter,
187      och med input.missing_data om inget värde är giltigt eller toppen inte är större än 0.
188      """
189      if (
190          wavelengths.shape != measured.shape
191          or known_wavelengths.shape != known_values.shape
192          or known_wavelengths.size < 2
193      ):
194          raise ValueError("input.invalid_argument: olika form eller färre än två kända punkter")
195      # Interpolation utan extrapolation: NaN utanför de kända punkterna. Se rev7, RX15.
196      known_at_lamp = np.interp(wavelengths, known_wavelengths, known_values, left=np.nan, right=np.nan)
197      valid = np.isfinite(measured) & np.isfinite(known_at_lamp) & (known_at_lamp > 0)
198      # Respons = uppmätt delat med känt, där det är giltigt. Se rev7, RX14.
199      ratio = np.full(measured.shape[0], np.nan)
200      np.divide(measured, known_at_lamp, out=ratio, where=valid)
201      if not np.any(valid):
202          raise ValueError("input.missing_data: inget giltigt värde i lampmätningen")
203      peak = np.max(ratio[valid])
204      if peak <= 0:
205          raise ValueError("input.missing_data: toppen är inte större än 0")
206      # Normerad: 1 där instrumentet är känsligast.
207      return ratio / peak
208
209
210  def apply_response(signal, response, mask, min_response):
211      """Delar signalen med responsen där masken är True och responsen är strikt större än min_response.
212
213      Parametrar: signal: (P,) float64 i counts; response: (P,) float64, relativ, utan enhet;
214      mask: (P,) bool, True där värdet duger; min_response: minsta giltiga respons, utan enhet.
215      Returnerar: (values, mask): values (P,) float64, relativ, NaN där det inte går att räkna, och den nya masken
             (P,) bool.
216      Fel: ValueError med input.invalid_argument vid olika form, en mask som inte är bool, en tröskel som inte är
217      ändlig och större än 0, eller NaN och inf på en giltig plats.
218      """
219      if signal.shape != response.shape:
220          raise ValueError(f"input.invalid_argument: response har annan form än signal {signal.shape}
                 {response.shape}")
221      if signal.shape != mask.shape:
222          raise ValueError(f"input.invalid_argument: mask har annan form än signal {signal.shape} {mask.shape}")
223      if mask.dtype != np.bool_:
224          raise ValueError(f"input.invalid_argument: mask måste vara en bool-array, fick {mask.dtype}")
225      if not np.isfinite(min_response) or min_response <= 0:
226          raise ValueError(f"input.invalid_argument: min_response måste vara ändligt och större än 0, fick
                 {min_response}")
227      _check("signal", signal[mask])
228      _check("response", response[mask])
229      # Strikt >: exakt min_response är ogiltig. Se rev7, RX16.
230      new_mask = mask & (response > min_response)
231      values = np.full(signal.shape[0], np.nan)
232      # Korrigerad signal = signal delad med responsen, bara där den nya masken är True.
233      np.divide(signal, response, out=values, where=new_mask)
234      return values, new_mask
```

```
235
236
237 def transmission_ratio(sample, reference, mask, min_reference):
238     """Delar provet med referensen där masken är True och referensen är strikt större än min_reference.
239
240     Parametrar: sample: provet, (P,) float64 i counts, får vara negativt; reference: referensen, (P,) float64 i
        counts;
241     mask: (P,) bool, True där värdet duger; min_reference: minsta giltiga referens i counts.
242     Returnerar: (values, mask): transmissionen (P,) float64 som kvot, NaN där den inte går att räkna,
243     och den nya masken (P,) bool.
244     Fel: ValueError med input.invalid_argument vid olika form, en mask som inte är bool, en tröskel som inte är
245     ändlig och större än 0, eller NaN och inf på en giltig plats.
246     """
247     if sample.shape != reference.shape:
248         raise ValueError(f"input.invalid_argument: reference har annan form än sample {sample.shape}
            {reference.shape}")
249     if sample.shape != mask.shape:
250         raise ValueError(f"input.invalid_argument: mask har annan form än sample {sample.shape} {mask.shape}")
251     if mask.dtype != np.bool_:
252         raise ValueError(f"input.invalid_argument: mask måste vara en bool-array, fick {mask.dtype}")
253     if not np.isfinite(min_reference) or min_reference <= 0:
254         raise ValueError(
255             f"input.invalid_argument: min_reference måste vara ändligt och större än 0, fick {min_reference}"
256         )
257     _check("sample", sample[mask])
258     _check("reference", reference[mask])
259     # Strikt >: exakt min_reference är ogiltig. Se rev7, T06.
260     valid = mask & (reference > min_reference)
261     values = np.full(sample.shape[0], np.nan)
262     # Transmission T = I_prov / I_ref, bara där den nya masken är True. Se rev7, RX17.
263     np.divide(sample, reference, out=values, where=valid)
264     return values, valid
```

2. spectrometer/source.py (83 rader, blob f93c8d8)

```
  1  """Bildkällan: en bildfil eller en webbkamera, lämnad som en ordbok. Filen räknar inte.
  2
  3  Används av main.py. Bilden lämnas som den är, (H, W, 3) uint8 i BGR.
  4  """
  5  import sys
  6  from pathlib import Path
  7
  8  import cv2
  9
 10
 11  def open_source(kind, value):
 12      """Öppnar en bildkälla.
 13
 14      Parametrar: kind: "file" eller "camera"; value: sökvägen till bildfilen (str eller Path),
 15      eller kamerans nummer N (int): N i /dev/videoN på Linux, 0 för första kameran på Windows.
 16      Returnerar: källan som en ordbok: {"kind": "file", "image": (H, W, 3) uint8} eller {"kind": "camera", "cap":
             VideoCapture}.
 17      Fel: ValueError med input.invalid_argument för okänd sort; RuntimeError med camera.open_failed om kameran
             eller
 18      filen inte går att öppna, filen inte är en bild eller inte har 8 bitar per kanal och formen (H, W, 3).
 19      """
 20      if kind == "camera":
 21          if sys.platform.startswith("linux"):
 22              backend = cv2.CAP_V4L2  # Video for Linux 2
 23          elif sys.platform == "win32":
 24              backend = cv2.CAP_DSHOW  # DirectShow på Windows
 25          else:
 26              backend = cv2.CAP_ANY  # OpenCV väljer själv
 27          message = f"camera.open_failed: kunde inte öppna /dev/video{value}. Kontrollera med v4l2-ctl
                 --list-devices."
 28          cap = None
 29          try:
 30              # Ett nummer som inte ryms i OpenCV:s heltal (till exempel 2147483648) ger cv2.error i stället för
                     False.
 31              cap = cv2.VideoCapture(value, backend)
 32              opened = cap.isOpened()
 33          except cv2.error as error:
 34              if cap is not None:
 35                  cap.release()
 36              raise RuntimeError(message) from error
 37          if not opened:
 38              cap.release()
 39              raise RuntimeError(message)
 40          return {"kind": "camera", "cap": cap}
 41      if kind != "file":
 42          raise ValueError(f"input.invalid_argument: Type is unknown {kind}")
 43      if not Path(value).is_file():
 44          raise RuntimeError(f"camera.open_failed: Image source does not exist {value}")
 45      img = cv2.imread(str(value), cv2.IMREAD_UNCHANGED)
 46      if img is None:
 47          raise RuntimeError(f"camera.open_failed: Could not open {value} as image")
 48      if img.dtype != "uint8":
 49          raise RuntimeError(f"camera.open_failed: Non-8bit per channel, {img.dtype}")
 50      if img.ndim == 2:
 51          img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
 52      elif img.ndim == 3 and img.shape[-1] == 4:
 53          img = cv2.cvtColor(img, cv2.COLOR_BGRA2BGR)
 54      if img.ndim != 3 or img.shape[-1] != 3:
 55          raise RuntimeError(f"camera.open_failed: Image is not (H, W, 3), shape is {img.shape}")
 56      return {"kind": kind, "image": img}
 57
 58
 59  def read_frame(src):
 60      """Ger en bild ur källan: en ny bild från kameran, eller en kopia av filens bild.
 61
 62      Parametrar: src: ordboken från open_source.
 63      Returnerar: bilden, (H, W, 3) uint8 i BGR och counts.
 64      Fel: RuntimeError med camera.read_failed om kameran inte ger någon bild.
 65      """
 66      if src["kind"] == "camera":
 67          ok, image = src["cap"].read()
 68          if not ok or image is None:
 69              raise RuntimeError("camera.read_failed: kunde inte läsa en bild från kameran")
 70          return image
 71      if src["kind"] == "file":
 72          return src["image"].copy()
 73
 74
 75  def close_source(src):
 76      """Stänger källan: släpper kameran; en fil behöver inte stängas.
 77
 78      Parametrar: src: ordboken från open_source.
 79      Returnerar: None
```

```
    Fel: inga.
    """
    if src["kind"] == "camera":
        src["cap"].release()
```

3. spectrometer/plot.py (26 rader, blob ff6b070)

```
  1  """Creates the window, does not compute"""
  2
  3  import matplotlib.pyplot as plt
  4
  5
  6  def open_plot():
  7      """Opens window with empty curve"""
  8
  9      plt.ion()
 10      fig, ax = plt.subplots()
 11
 12      lines = ax.plot([], [])
 13      line = lines[0]
 14
 15      return {"fig": fig, "ax": ax, "line": line}
 16
 17  def draw(win, x, values, x_label, y_label):
 18      """Changes the values of the curve and redraws"""
 19
 20      win["line"].set_data(x, values)
 21      win["ax"].relim()
 22      win["ax"].autoscale_view()
 23      win["ax"].set_xlabel(x_label)
 24      win["ax"].set_ylabel(y_label)
 25
 26      plt.pause(0.001)
```

4. spectrometer/files.py (132 rader, blob 01b129d)

```
"""Sparar mätningar på disk och läser punktfiler och kända fördelningar. Filen räknar inte.

Används av main.py (save, calibrate och response build).
"""
import csv
import json
import math
from datetime import datetime
from pathlib import Path

import cv2
import numpy as np


def save_measurement(parent, spectrum, image, settings):
    """Sparar en mätning som en mapp med spectrum.csv, raw.png och settings.json.

    Parametrar: parent: föräldramappen (str eller Path); spectrum: ordboken från measure med x, values, mask,
    x_label och y_label; image: råbilden, (H, W, 3) uint8; settings: inställningarna (bara tal, text, tuple,
    listor, bool och None).
    Returnerar: mätmappen som Path, till exempel data/20261016-153012 eller data/20261016-153012-2.
    Fel: OSError om något inte går att skriva, med files.write_failed om raw.png inte sparas.
    """
    stamp = datetime.now().strftime("%Y%m%d-%H%M%S")
    folder = Path(parent) / stamp
    counter = 2
    while folder.exists():
        folder = Path(parent) / f"{stamp}-{counter}"
        counter += 1
    folder.mkdir(parents=True)
    transmission = settings.get("mode") == "transmission"  # valfritt (0.9.3): kolumnen absorbance
    with open(folder / "spectrum.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        x_col = "x_px" if spectrum["x_label"] == "pixel" else "x_nm"
        header = [x_col, "value", "valid"]
        if transmission:
            header.append("absorbance")
        writer.writerow(header)
        for x, value, ok in zip(spectrum["x"], spectrum["values"], spectrum["mask"]):
            cell = f"{value:.10g}" if ok else ""
            row = [f"{x:.10g}", cell, "true" if ok else "false"]
            if transmission:
                # Absorbans A = -log10(T), tom där T saknas eller är 0 eller mindre. Se rev7, Krav 22.
                row.append(f"{-math.log10(value):.10g}" if ok and value > 0 else "")
            writer.writerow(row)
    raw_path = folder / "raw.png"
    written = cv2.imwrite(str(raw_path), image)
    if not written:
        raise OSError(f"files.write_failed: raw.png kunde inte sparas: {raw_path}")
    with open(folder / "settings.json", "w", encoding="utf-8") as f:
        json.dump(settings, f, indent=2, ensure_ascii=False)
    return folder


def load_points(path):
    """
    Läser en punktfil med kolumnerna pixel, wavelength_nm och check.

    Parametrar: path: sökvägen till CSV-filen (str eller Path); check är 0 för anpassning och 1 för kontroll
    Returnerar: (pixels, wavelengths, is_check): pixlar i hela bilden (K,) float64, våglängder i nm (K,) float64
        och is_check (K,) bool, True för en kontrollpunkt.
    Fel: ValueError med input.invalid_argument, filnamnet och radnumret vid fel rubrik, fel antal fält, ett fält
        som inte är ett tal, ett ogiltigt värde eller check som inte är 0 eller 1; OSError om filen saknas.
    """
    name = Path(path).name
    pixels = []
    wavelengths = []
    is_check = []
    with open(path, encoding="utf-8-sig", newline="") as f:
        for line_no, row in enumerate(csv.reader(f), start=1):
            if not row:
                continue  # tom rad, en sist i filen är vanlig
            if line_no == 1:
                if row != ["pixel", "wavelength_nm", "check"]:
                    raise ValueError(f"input.invalid_argument: {name} rad {line_no}: fel rubrik, väntade
                        pixel,wavelength_nm,check")
                continue
            if len(row) != 3:
                raise ValueError(f"input.invalid_argument: {name} rad {line_no}: fel antal fält, väntade 3")
            try:
                pixel = float(row[0])
            except ValueError as err:
                raise ValueError(f"input.invalid_argument: {name} rad {line_no}: pixel är inte ett tal") from
```

```
                    err
            try:
                wavelength = float(row[1])
            except ValueError as err:
                raise ValueError(f"input.invalid_argument: {name} rad {line_no}: wavelength_nm är inte ett tal")
                    from err
            if not math.isfinite(pixel) or pixel < 0:
                raise ValueError(f"input.invalid_argument: {name} rad {line_no}: pixel har ett ogiltigt värde")
            if row[2] not in ("0", "1"):
                raise ValueError(f"input.invalid_argument: {name} rad {line_no}: check ska vara 0 eller 1")
            pixels.append(pixel)
            wavelengths.append(wavelength)
            is_check.append(row[2] == "1")
    pixels = np.array(pixels, dtype=np.float64)
    wavelengths = np.array(wavelengths, dtype=np.float64)
    is_check = np.array(is_check, dtype=bool)
    return pixels, wavelengths, is_check


def load_known(path):
    """Läser en känd fördelning med kolumnerna wavelength_nm och relative_power_per_nm.

    Parametrar: path: sökvägen till CSV-filen (str eller Path).
    Returnerar: (wavelengths, values): våglängder i stigande nm (K,) float64 och relativ effekt per nm (K,)
        float64.
    Fel: ValueError med input.invalid_argument, filnamnet och radnumret vid fel rubrik, fel antal fält, ett fält
        som
    inte är ett tal, ett ogiltigt värde, en våglängd som inte stiger eller färre än två datarader; OSError om
        filen saknas.
    """
    name = Path(path).name
    wavelengths = []
    values = []
    with open(path, encoding="utf-8-sig", newline="") as f:
        for line_no, row in enumerate(csv.reader(f), start=1):
            if not row:
                continue  # tom rad, en sist i filen är vanlig
            if line_no == 1:
                if row != ["wavelength_nm", "relative_power_per_nm"]:
                    raise ValueError(f"input.invalid_argument: {name} rad {line_no}: fel rubrik {row}")
                continue
            if len(row) != 2:
                raise ValueError(f"input.invalid_argument: {name}: fel antal fält, rad {line_no}")
            try:
                wavelength = float(row[0])
                power = float(row[1])
            except ValueError as err:
                raise ValueError(f"input.invalid_argument: {name}: fältet är inte ett tal, rad {line_no}") from
                    err
            if not (math.isfinite(wavelength) and math.isfinite(power)) or power < 0:
                raise ValueError(f"input.invalid_argument: {name}: ogiltigt värde, rad {line_no}")
            if wavelengths and wavelength <= wavelengths[-1]:
                raise ValueError(f"input.invalid_argument: {name}: våglängden stiger inte, rad {line_no}")
            wavelengths.append(wavelength)
            values.append(power)
    if len(wavelengths) < 2:
        raise ValueError(f"input.invalid_argument: färre än två datarader i {name}")
    return np.array(wavelengths, dtype=np.float64), np.array(values, dtype=np.float64)
```

5. spectrometer/main.py (682 rader, blob e3680fe)

```
"""Håller ihop programmet: startas med python -m spectrometer.main och styrs med kommandon vid prompten spektro>."""
import argparse
import shlex
import sys

import matplotlib.pyplot as plt
import numpy as np

from spectrometer import calc, files, plot, source

DEFAULTS = {
    "source": ("camera", 0),  # ("file", sökväg) eller ("camera", N)
    "roi": None,  # None är hela bilden, annars (x, y, bredd, höjd) i pixlar
    "axis": "horizontal",  # riktningen spektrumet läses längs
    "channel": "gray",  # färgkanal: gray, sum, red, green eller blue
    "output_dir": "data",  # föräldramappen för mätmapparna
    "frames": 1,  # bilder per mätning
    "min_response": 0.01,  # minsta respons, utan enhet
    "min_reference": 1.0,  # minsta referens i counts
}

CAPTURE_FRAMES = 16  # bilder i dark och reference: medlet av flera bilder har mindre brus än en enda

AXES = ("horizontal", "vertical")


def new_state(settings):
    """Ger en ny state-ordbok med en kopia av inställningarna.

    Parametrar: settings: inställningarna, till exempel DEFAULTS (ändras inte).
    Returnerar: state, en ordbok med nycklarna nedan.
    Fel: inga.
    """
    return {
        "settings": settings.copy(),  # inställningarna, skrivs till settings.json
        "image": None,  # senaste råbilden, (H, W, 3) uint8 i counts, None tills step
        "spectrum": None,  # senaste mätningen (ordbok från measure), None tills step
        "coefficients": None,  # kalibreringens koefficienter, float64, a0 först (calibrate)
        "pixel_range": None,  # kalibreringens giltiga intervall (låg, hög) i pixlar (calibrate)
        "dark": None,  # mörkret, (H, W, 3) float64 i counts (dark)
        "response": None,  # responsen: {"x": nm, "values": relativ} (response build)
        "use_response": False,  # True när responsen ska användas (response on och off)
        "reference": None,  # referensspektrumet med kanal, values i counts (reference)
        "mode": "direct",  # "direct" eller "transmission" (mode)
    }


def measure(state, src=None):
    """Mäter ett spektrum med inställningarna och lägger det och råbilden i state.

    Ordning: läs frames bilder, dra av mörkret, ROI, kanal och medel, nm, respons, transmission.

    Parametrar: state: programmets state; src: en redan öppen källa som lånas (start), eller None för att öppna och
    stänga källan här.
    Returnerar: spektrumet, en ordbok med x (P,) float64 i pixlar eller nm, values (P,) float64, mask (P,) bool,
    x_label och y_label.
    Fel: ValueError med input.invalid_argument eller input.missing_data, RuntimeError med camera.open_failed eller
    camera.read_failed.
    """
    cfg = state["settings"]
    kind, value = cfg["source"]
    frames = cfg["frames"]
    spectra = []
    opened_here = src is None
    if opened_here:
        src = source.open_source(kind, value)
    try:
        for i in range(frames):
            image = source.read_frame(src)
            if state["dark"] is not None:
                corrected = image.astype(np.float64)
                corrected = calc.subtract_dark(corrected, state["dark"])
            else:
                corrected = image
            roi = cfg["roi"]
            if roi is None:
                roi = (0, 0, image.shape[1], image.shape[0])
            table = calc.prepare(corrected, roi, cfg["axis"], cfg["channel"])
            values = calc.extract_spectrum(table)
            spectra.append(values)
```

```
    finally:
        if opened_here:
            source.close_source(src)
    stack = np.stack(spectra)
    mean, _stddev = calc.average_spectra(stack)
    if cfg["axis"] == "horizontal":
        first, count = roi[0], roi[2]
    else:
        first, count = roi[1], roi[3]
    x = np.arange(first, first + count).astype("float64")
    values = mean
    mask = np.isfinite(values)
    x_label = "pixel"
    y_label = "signal (counts)"
    if state["coefficients"] is not None:
        low, high = state["pixel_range"]
        inside = (x >= low) & (x <= high)
        mask = mask & inside
        x = calc.evaluate_wavelength(x, state["coefficients"])
        x_label = "våglängd (nm)"
    if x[0] > x[-1]:
        x = x[::-1]
        values = values[::-1]
        mask = mask[::-1]
    if state["use_response"] and state["mode"] == "direct":
        if state["coefficients"] is None:
            raise ValueError("input.missing_data: ingen kalibrering, kör calibrate först")
        response_at_x = np.interp(x, state["response"]["x"], state["response"]["values"], left=np.nan,
            right=np.nan)
        mask = mask & np.isfinite(response_at_x)
        values, mask = calc.apply_response(values, response_at_x, mask, cfg["min_response"])
        y_label = "korrigerad signal (relativ)"
    spectrum = {"x": x, "values": values, "mask": mask, "x_label": x_label, "y_label": y_label}
    if state["mode"] == "transmission":
        if state["reference"] is None:
            raise ValueError("input.missing_data: ingen referens, kör reference först")
        if not np.array_equal(spectrum["x"], state["reference"]["x"]):
            raise ValueError(
                "input.invalid_argument: x-axeln skiljer sig från referensens: "
                "ROI eller kalibrering ändrades, kör reference igen"
            )
        if cfg["channel"] != state["reference"]["channel"]:
            raise ValueError(
                f"input.invalid_argument: kanalen {cfg['channel']} skiljer sig från referensens "
                f"{state['reference']['channel']}, kör reference igen"
            )
        spectrum["values"], spectrum["mask"] = calc.transmission_ratio(
            spectrum["values"], state["reference"]["values"], spectrum["mask"] & state["reference"]["mask"],
            state["settings"]["min_reference"],
        )
        spectrum["y_label"] = "transmission (kvot)"
    state["image"] = image
    state["spectrum"] = spectrum
    return spectrum


def _measure_direct(state, n):
    """Mäter med en kopia av state i mode direct, utan respons och med n bilder (lampa och referens).

    Parametrar: state: programmets state (ändras inte); n: antalet bilder, minst 1.
    Returnerar: spektrumet från measure, i counts.
    Fel: samma som measure.
    """
    direct_state = state.copy()
    direct_state["settings"] = state["settings"].copy()
    direct_state["mode"] = "direct"
    direct_state["use_response"] = False
    direct_state["settings"]["frames"] = n
    return measure(direct_state)


def _to_int(name, text, smallest):
    """Gör om ett ord till ett heltal som är minst smallest.

    Parametrar: name: namnet i felmeddelandet (str); text: ordet från prompten (str); smallest: minsta tillåtna
        tal (int).
    Returnerar: talet (int).
    Fel: ValueError med input.invalid_argument om ordet inte är ett heltal eller är mindre än smallest.
    """
    try:
        number = int(text)
    except ValueError as error:
        raise ValueError(f"input.invalid_argument: {name} ska vara ett heltal, fick {text}") from error
    if number < smallest:
        raise ValueError(f"input.invalid_argument: {name} ska vara minst {smallest}")
```

```
    return number


def _need(args, names, command):
    """Kontrollerar att kommandot fick lika många ord som names.

    Parametrar: args: orden efter kommandot (list); names: namnen på orden som behövs (tuple); command:
        kommandots namn (str).
    Returnerar: None
    Fel: ValueError med input.invalid_argument när ett ord saknas (felet säger vilket) eller när orden är för
        många.
    """
    if len(args) < len(names):
        missing = names[len(args)]
        raise ValueError(f"input.invalid_argument: {command}: {missing} saknas")
    elif len(args) > len(names):
        raise ValueError(f"input.invalid_argument: {command}: för många ord, väntade {len(names)}")


def _set(state, key, value) -> None:
    """Lägger en inställning i state och glömmer den gamla mätningen (roi och channel glömmer också responsen).

    Parametrar: state: programmets state; key: nyckeln i settings (str); value: det nya värdet.
    Returnerar: None
    Fel: inga.
    """
    state["settings"][key] = value
    state["spectrum"] = None
    if key in ("roi", "channel") and state["response"] is not None:
        state["response"] = None
        state["use_response"] = False
        print("Responsen glömd. Kör response build igen.")


def cmd_help(state, args):
    """Kommandot help: skriver en rad per kommando ur COMMANDS.

    Parametrar: state: programmets state; args: orden efter kommandot, inga.
    Returnerar: None
    Fel: ValueError med input.invalid_argument vid ord efter help.
    """
    _need(args, (), "help")
    for name, entry in COMMANDS.items():
        _function, help_text = entry
        print(name, help_text)


def cmd_quit(state, args):
    """Kommandot quit: skriver Avslutar. och ber main att avsluta.

    Parametrar: state: programmets state; args: orden efter kommandot, inga.
    Returnerar: True
    Fel: ValueError med input.invalid_argument vid ord efter quit.
    """
    _need(args, (), "quit")
    print("Avslutar.")
    return True


def cmd_show(state, args):
    """Kommandot show: skriver inställningarna, kalibrering, mörker, respons, referens, mode och om en mätning
        finns.

    Parametrar: state: programmets state; args: orden efter kommandot, inga.
    Returnerar: None
    Fel: ValueError med input.invalid_argument vid ord efter show.
    """
    _need(args, (), "show")
    for key, value in state["settings"].items():
        print(f"{key}: {value}")
    calibration = "ja" if state["coefficients"] is not None else "nej"
    dark = "ja" if state["dark"] is not None else "nej"
    response = "ja" if state["response"] is not None else "nej"
    reference = "ja" if state["reference"] is not None else "nej"
    use = "on" if state["use_response"] else "off"
    print(f"kalibrering: {calibration}")
    print(f"mörker: {dark}")
    print(f"respons: {response}, {use}")
    print(f"referens: {reference}")
    print(f"mode: {state['mode']}")
    if state["spectrum"] is not None:
        print("Mätning: ja")
    else:
        print("Mätning: nej")

```

```
246
247 def cmd_source(state, args):
248     """Kommandot source file PATH eller source camera N: byter källa och glömmer den gamla bilden.
249
250     Parametrar: state: programmets state; args: ["file", PATH] eller ["camera", N].
251     Returnerar: None
252     Fel: ValueError med input.invalid_argument om ord saknas, sorten är okänd eller N inte är ett heltal minst
            0.
253     """
254     if not args:
255         raise ValueError("input.invalid_argument: source: kind saknas")
256     if args[0] == "file":
257         _need(args, ("kind", "path"), "source")
258         _set(state, "source", ("file", args[1]))
259     elif args[0] == "camera":
260         _need(args, ("kind", "index"), "source")
261         index = _to_int("index", args[1], 0)
262         _set(state, "source", ("camera", index))
263     else:
264         raise ValueError(f"input.invalid_argument: okänd källa {args[0]}, välj file eller camera")
265     state["image"] = None
266     print(f"source: {state['settings']['source']}")
267
268
269 def cmd_roi(state, args):
270     """Kommandot roi X Y W H eller roi full: sätter ROI:n i pixlar efter att ha kontrollerat den.
271
272     Parametrar: state: programmets state; args: ["full"] eller fyra heltal x, y, width och height.
273     Returnerar: None
274     Fel: ValueError med input.invalid_argument om ord saknas, inte är heltal, är för små eller ROI:n inte ryms
275     i den senaste bilden. Den gamla ROI:n står då kvar.
276     """
277     if args == ["full"]:
278         _set(state, "roi", None)
279         print("roi: None (hela bilden)")
280         return
281     _need(args, ("x", "y", "width", "height"), "roi")
282     x = _to_int("x", args[0], 0)
283     y = _to_int("y", args[1], 0)
284     width = _to_int("width", args[2], 1)
285     height = _to_int("height", args[3], 1)
286     new_roi = (x, y, width, height)
287     if state["image"] is not None:
288         calc.prepare(state["image"], new_roi, state["settings"]["axis"], state["settings"]["channel"])
289     else:
290         print("ROI:n kontrolleras vid nästa mätning.")
291     _set(state, "roi", new_roi)
292     print(f"roi: {new_roi}")
293
294
295 def cmd_channel(state, args):
296     """Kommandot channel NAMN: väljer färgkanal ur calc.CHANNELS.
297
298     Parametrar: state: programmets state; args: ett namn.
299     Returnerar: None
300     Fel: ValueError med input.invalid_argument om namnet saknas eller är okänt.
301     """
302     _need(args, ("name",), "channel")
303     if args[0] not in calc.CHANNELS:
304         allowed = ", ".join(calc.CHANNELS)
305         raise ValueError(f"input.invalid_argument: okänd kanal {args[0]}, välj {allowed}")
306     _set(state, "channel", args[0])
307     print(f"channel: {args[0]}")
308
309
310 def cmd_axis(state, args):
311     """Kommandot axis RIKTNING: väljer riktning ur AXES.
312
313     Parametrar: state: programmets state; args: ett namn.
314     Returnerar: None
315     Fel: ValueError med input.invalid_argument om namnet saknas eller är okänt.
316     """
317     _need(args, ("name",), "axis")
318     if args[0] not in AXES:
319         allowed = ", ".join(AXES)
320         raise ValueError(f"input.invalid_argument: okänd riktning {args[0]}, välj {allowed}")
321     _set(state, "axis", args[0])
322     print(f"axis: {args[0]}")
323
324
325 def cmd_frames(state, args):
326     """Kommandot frames N: antalet bilder per mätning.
327
328     Parametrar: state: programmets state; args: ett heltal N, minst 1.
329     Returnerar: None
```

```
    Fel: ValueError med input.invalid_argument om N saknas, inte är ett heltal eller är mindre än 1.
    """
    _need(args, ("N",), "frames")
    n = _to_int("N", args[0], 1)
    _set(state, "frames", n)
    print(f"frames: {n}")


def cmd_step(state, args):
    """Kommandot step: mäter, skriver toppen och antalet bilder, och varnar vid mättnad och transmission över 1.

    Parametrar: state: programmets state; args: orden efter kommandot, inga.
    Returnerar: None
    Fel: samma som measure, och ValueError med input.invalid_argument vid ord efter step.
    """
    _need(args, (), "step")
    spectrum = measure(state)
    i = np.nanargmax(spectrum["values"])
    word = "bild" if state["settings"]["frames"] == 1 else "bilder"
    print(f"Topp {spectrum["values"][i]:g} vid {spectrum["x_label"]} {spectrum["x"][i]:g}
        ({state["settings"]["frames"]} {word})")
    crop = state["image"]
    if state["settings"]["roi"] is not None:
        crop = state["image"][state["settings"]["roi"][1]:state["settings"]["roi"][1] +
            state["settings"]["roi"][3], state["settings"]["roi"][0]:state["settings"]["roi"][0] +
            state["settings"]["roi"][2]]
    if crop.max() >= 255:
        print("Varning: mättad (255 i råbilden). Sänk exponeringen.")
    if state["mode"] == "transmission":
        if np.any(spectrum["values"][spectrum["mask"]] > 1):
            over = np.count_nonzero(spectrum["values"][spectrum["mask"]] > 1)
            unit = "position" if over == 1 else "positioner"
            print(f"Varning: transmission över 100 % på {over} {unit}")


def cmd_save(state, args):
    """Kommandot save [DIR]: sparar senaste mätningen i en ny mätmapp.

    Parametrar: state: programmets state; args: inga ord (output_dir) eller en föräldramapp.
    Returnerar: None
    Fel: ValueError med input.invalid_argument vid fler än ett ord och input.missing_data om ingen mätning
        finns;
    OSError om något inte går att skriva.
    """
    if len(args) > 1:
        raise ValueError("input.invalid_argument: save tar högst ett ord (DIR)")
    if state["spectrum"] is None:
        raise ValueError("input.missing_data: Ingen mätning än. Kör step först.")
    parent = args[0] if args else state["settings"]["output_dir"]
    saved = state["settings"].copy()
    saved["coefficients"] = state["coefficients"].tolist() if state["coefficients"] is not None else None
    saved["dark"] = state["dark"] is not None
    saved["use_response"] = state["use_response"]
    saved["mode"] = state["mode"]
    folder = files.save_measurement(parent, state["spectrum"], state["image"], saved)
    print(f"Sparad i {folder.as_posix()}")


def cmd_start(state, args):
    """Kommandot start: visar spektrumet live tills Ctrl+C eller tills fönstret stängs.

    Parametrar: state: programmets state; args: orden efter kommandot, inga.
    Returnerar: None
    Fel: samma som measure; källan och fönstret stängs alltid i finally.
    """
    _need(args, (), "start")
    print("Live. Tryck Ctrl+C eller stäng fönstret för att stoppa.")
    kind, value = state["settings"]["source"]
    src = None
    win = None
    try:
        src = source.open_source(kind, value)
        win = plot.open_plot()
        while True:
            if not plt.fignum_exists(win["fig"].number):
                print("live stoppad")
                break
            spectrum = measure(state, src)
            plot.draw(win, spectrum["x"], np.where(spectrum["mask"], spectrum["values"], np.nan),
                spectrum["x_label"], spectrum["y_label"])
            plt.pause(0.05)
    except KeyboardInterrupt:
        print("live stoppad")
    finally:
        if src is not None:
```

```
            source.close_source(src)
        if win is not None:
            plt.close(win["fig"])


def cmd_calibrate(state, args):
    """Kommandot calibrate PATH [DEGREE]: anpassar våglängden till en punktfil och prövar den mot kontrollpunkterna.

    Parametrar: state: programmets state; args: punktfilens sökväg och valfritt graden 1 eller 2.
    Returnerar: None
    Fel: ValueError med input.invalid_argument vid fel ord, fel i punktfilen, för få punkter eller en linje som inte
    är monoton i det giltiga intervallet; OSError om filen saknas. Den gamla kalibreringen står då kvar.
    """
    if not args:
        _need(args, ("PATH",), "calibrate")  # kommatecknet är obligatoriskt
    if len(args) > 2:
        raise ValueError("input.invalid_argument: calibrate tar högst två ord (PATH DEGREE)")
    if len(args) == 2:
        degree = _to_int("DEGREE", args[1], 1)
    else:
        degree = 1
    pixels, wavelengths, is_check = files.load_points(args[0])
    fit_pixels = pixels[~is_check]
    check_pixels = pixels[is_check]
    fit_wavelengths = wavelengths[~is_check]
    check_wavelengths = wavelengths[is_check]
    coefficients = calc.fit_wavelength(fit_pixels, fit_wavelengths, degree)  # bara anpassningspunkter
    print(f"a0: {coefficients[0]:g}")
    print(f"a1: {coefficients[1]:g}")
    if degree == 2:
        print(f"a2: {coefficients[2]:g}")
    width = np.max(fit_pixels) - np.min(fit_pixels)
    valid_range = (max(np.min(fit_pixels) - width, 0), np.max(fit_pixels) + width)
    print(f"anpassat: pixel {np.min(fit_pixels):g} till {np.max(fit_pixels):g}")
    print(f"giltigt: pixel {valid_range[0]:g} till {valid_range[1]:g}")
    # Linjen måste vara monoton i det giltiga intervallet, annars får två pixlar samma våglängd.
    test_pixels = np.arange(valid_range[0], valid_range[1] + 1, dtype=np.float64)
    diffs = np.diff(calc.evaluate_wavelength(test_pixels, coefficients))
    if not (np.all(diffs > 0) or np.all(diffs < 0)):
        raise ValueError(
            f"input.invalid_argument: inte monoton mellan pixel {valid_range[0]:g} och {valid_range[1]:g}"
        )
    residuals = fit_wavelengths - calc.evaluate_wavelength(fit_pixels, coefficients)
    for residual in residuals:
        print(f"residual: {residual:g}")
    check_errors = check_wavelengths - calc.evaluate_wavelength(check_pixels, coefficients)
    # Lokalt steg: hur många nm en pixel är vid kontrollpunkten.
    step = np.abs(calc.evaluate_wavelength(check_pixels + 1, coefficients) -
        calc.evaluate_wavelength(check_pixels, coefficients))
    # varna när abs(fel) är större än steget
    for pixel, error, s in zip(check_pixels, check_errors, step):
        print(f"kontrollfel vid pixel {pixel:g}: {error:g}")
        if abs(error) > s:
            print(f"Varning: kontrollfelet {error:g} nm är större än steget {s:g} nm.")
    if check_pixels.size == 0:
        print("inte kontrollerad: inga kontrollpunkter i filen")
    state["coefficients"] = coefficients
    low, high = valid_range  # packa upp först
    state["pixel_range"] = (low, high)
    state["spectrum"] = None


def cmd_dark(state, args):
    """Kommandot dark [N]: mäter mörkret som medel av N bilder med blockerad ljusväg.

    Parametrar: state: programmets state; args: inga ord (CAPTURE_FRAMES bilder) eller ett heltal N, minst 1.
    Returnerar: None
    Fel: ValueError med input.invalid_argument vid fel ord, RuntimeError från källan. Det gamla mörkret står då kvar.
    """
    if len(args) > 1:
        raise ValueError("input.invalid_argument: dark tar högst ett ord (N)")
    if args:
        N = _to_int("N", args[0], 1)
    else:
        N = CAPTURE_FRAMES
    kind, value = state["settings"]["source"]
    src = source.open_source(kind, value)
    flat = []
    try:
        for _ in range(N):
            frame = source.read_frame(src).astype(np.float64)
            shape = frame.shape
```

```
491             flat.append(frame.reshape(-1))
492     finally:
493         source.close_source(src)
494     stacked = np.stack(flat)
495     # Mörkret är medlet av N bilder, pixel för pixel och kanal för kanal. Se rev7, RX03.
496     mean_image, _spread = calc.average_spectra(stacked)
497     state["dark"] = mean_image.reshape(shape)
498     state["spectrum"] = None
499     word = "bild" if N == 1 else "bilder"
500     print(f"mörker: {N} {word}, medel {np.mean(state["dark"]):g}")
501     if state["image"] is not None and np.mean(state["dark"]) >= 0.5 * np.mean(state["image"]):
502         print("Varning: mörkret liknar bilden. Var ljusvägen blockerad?")
503     if state["reference"] is not None:
504         # Referensen mättes med det gamla mörkret, så den gäller inte längre.
505         state["reference"] = None
506         print("referensen togs bort: kör reference igen")
507
508
509 def cmd_response(state, args):
510     """Kommandot response build KNOWNFILE, response on eller response off.
511
512     Parametrar: state: programmets state; args: ["build", KNOWNFILE], ["on"] eller ["off"].
513     Returnerar: None
514     Fel: ValueError med input.invalid_argument vid fel ord och input.missing_data utan mörker, kalibrering eller
            respons;
515     OSError om filen saknas.
516     """
517     if not args or args[0] not in ("build", "on", "off"):
518         raise ValueError("input.invalid_argument: första ordet ska vara build, on eller off")
519     if args[0] == "build":
520         rest = args[1:]
521         _need(rest, ("KNOWNFILE",), "response")
522         if state["dark"] is None:
523             raise ValueError("input.missing_data: inget mörker, kör dark först")
524         if state["coefficients"] is None:
525             raise ValueError("input.missing_data: ingen kalibrering, kör calibrate först")
526         known_wavelengths, known_values = files.load_known(rest[0])
527         lamp = _measure_direct(state, CAPTURE_FRAMES)
528         lamp_values = np.where(lamp["mask"], lamp["values"], np.nan)
529         response_values = calc.build_response(lamp["x"], lamp_values, known_wavelengths, known_values)
530         state["response"] = {"x": lamp["x"].copy(), "values": response_values}
531         state["spectrum"] = None
532         print(f"respons: {np.count_nonzero(np.isfinite(state["response"]["values"]))} av
                {len(state["response"]["values"])} positioner giltiga")
533     elif args[0] == "on":
534         if state["response"] is None:
535             raise ValueError("input.missing_data: ingen respons, kör response build först")
536         state["use_response"] = True
537         state["spectrum"] = None
538         print("respons: on")
539     elif args[0] == "off":
540         state["use_response"] = False
541         state["spectrum"] = None
542         print("respons: off")
543
544
545 def cmd_reference(state, args):
546     """Kommandot reference [N]: mäter referensen utan prov, utan respons och transmission, och sparar den med
            kanalen.
547
548     Parametrar: state: programmets state; args: inga ord (CAPTURE_FRAMES bilder) eller ett heltal N, minst 1.
549     Returnerar: None
550     Fel: ValueError med input.invalid_argument vid fel ord och input.missing_data utan mörker; samma som
            measure.
551     """
552     if state["dark"] is None:
553         raise ValueError("input.missing_data: inget mörker, kör dark först")
554     if len(args) > 1:
555         raise ValueError("input.invalid_argument: reference tar högst ett ord (N)")
556     if args:
557         N = _to_int("N", args[0], 1)
558     else:
559         N = CAPTURE_FRAMES
560     spectrum = _measure_direct(state, N)
561     spectrum["channel"] = state["settings"]["channel"]
562     state["reference"] = spectrum
563     state["spectrum"] = None
564     word = "bild" if N == 1 else "bilder"
565     print(f"Referens: {N} {word}, x från {spectrum["x"][0]:g} till {spectrum["x"][-1]:g}")
566
567
568 def cmd_mode(state, args):
569     """Kommandot mode direct eller mode transmission.
570
571     Parametrar: state: programmets state; args: ["direct"] eller ["transmission"].
```

```
572      Returnerar: None
573      Fel: ValueError med input.invalid_argument vid fel ord och input.missing_data för transmission utan
             referens.
574      """
575      if len(args) != 1 or args[0] not in ("direct", "transmission"):
576          raise ValueError("input.invalid_argument: mode tar ett ord, direct eller transmission")
577      if args[0] == "transmission" and state["reference"] is None:
578          raise ValueError("input.missing_data: ingen referens, kör reference först")
579      state["mode"] = args[0]
580      state["spectrum"] = None
581      print(f"mode: {args[0]}")
582
583
584  COMMANDS = {
585      "help": (cmd_help, "Visa kommandona"),
586      "quit": (cmd_quit, "Avsluta"),
587      "show": (cmd_show, "Visa inställningarna"),
588      "source": (cmd_source, "file PATH eller camera N"),
589      "roi": (cmd_roi, "X Y W H i pixlar, eller full för hela bilden"),
590      "channel": (cmd_channel, "gray, sum, red, green eller blue"),
591      "axis": (cmd_axis, "horizontal eller vertical"),
592      "frames": (cmd_frames, "N: bilder per mätning"),
593      "step": (cmd_step, "mät och skriv toppen"),
594      "save": (cmd_save, "[DIR]: spara senaste mätningen"),
595      "start": (cmd_start, "live med kameran, Ctrl+C stoppar"),
596      "calibrate": (cmd_calibrate, "PATH [DEGREE]: anpassa våglängd till en punktfil"),
597      "dark": (cmd_dark, "[N]: mät mörker med blockerad ljusväg"),
598      "response": (cmd_response, "build KNOWNFILE | on | off"),
599      "reference": (cmd_reference, "[N]: mät referens utan prov"),
600      "mode": (cmd_mode, "direct|transmission"),
601  }
602
603
604  def run_command(state, line):
605      """Delar en rad i ord och anropar kommandot.
606
607      Parametrar: state: programmets state; line: raden från prompten (str).
608      Returnerar: svaret från kommandot: True från quit, annars None (också för en tom rad).
609      Fel: ValueError med input.invalid_argument vid citattecken utan slut eller okänt kommando; kommandots egna
             fel.
610      """
611      try:
612          words = shlex.split(line)
613      except ValueError as error:
614          raise ValueError("input.invalid_argument: citattecken saknar slut") from error
615      if not words:
616          return None  # tom rad: bara en ny prompt
617      if words[0] not in COMMANDS:
618          raise ValueError(f"input.invalid_argument: okänt kommando {words[0]}. Skriv help för en lista.")
619      function, _help_text = COMMANDS[words[0]]
620      return function(state, words[1:])
621
622
623  def run_file(state, path):
624      """Mäter en bildfil en gång, som main gjorde i 0.3: skriver toppen, sparar mätningen och ritar spektrumet.
625
626      Parametrar: state: programmets state; path: sökvägen till bildfilen (str).
627      Returnerar: slutkoden, 0 om det gick och 1 vid ett fel (en rad Fel:).
628      Fel: fångar ValueError, RuntimeError och OSError.
629      """
630      state["settings"]["source"] = ("file", path)
631      try:
632          spectrum = measure(state)
633          i = np.argmax(spectrum["values"])
634          print(f"Topp {spectrum["values"][i]:g} vid pixel {spectrum["x"][i]:g}")
635
636          folder = files.save_measurement(state["settings"]["output_dir"], spectrum, state["image"],
                       state["settings"])
637          print(f"Sparad i {folder.as_posix()}")
638
639          win = plot.open_plot()
640          plot.draw(win, spectrum["x"], np.where(spectrum["mask"], spectrum["values"], np.nan),
                       spectrum["x_label"], spectrum["y_label"])
641          plt.show(block=True)
642
643      except (ValueError, RuntimeError, OSError) as err:
644          print(f"Fel: {err}")
645          return 1
646      return 0
647
648
649  def main():
650      """Kör prompten spektro> tills quit, Ctrl+D eller Ctrl+C vid prompten; ett fel blir en rad Fel:.
651
652      Med en sökväg efter kommandot mäts bara den bildfilen en gång, som i 0.3 (run_file).
```

```
653      Parametrar: inga (sökvägen läses av argparse).
654      Returnerar: None för prompten (sys.exit ger då koden 0), annars slutkoden från run_file.
655      Fel: fångar ValueError, RuntimeError och OSError per rad, och KeyboardInterrupt under ett kommando
             (Avbrutet.).
656      """
657      parser = argparse.ArgumentParser(prog="python -m spectrometer.main", description="Utan sökväg: prompten
             spektro>. Med sökväg: läser bildfilen, skriver toppen, sparar mätningen och ritar spektrumet.")
658      parser.add_argument("path", nargs="?", help="sökvägen till en bildfil (utan den startar prompten)")
659      args = parser.parse_args()
660
661      state = new_state(DEFAULTS)
662      if args.path is not None:
663          return run_file(state, args.path)
664      print("Skriv help för kommandona. quit eller Ctrl+D avslutar.")
665      while True:
666          try:
667              line = input("spektro> ")
668          except (EOFError, KeyboardInterrupt):
669              print()
670              break
671          try:
672              answer = run_command(state, line)
673              if answer:
674                  break
675          except (ValueError, RuntimeError, OSError) as error:
676              print(f"Fel: {error}")
677          except KeyboardInterrupt:
678              print("Avbrutet.")
679
680
681  if __name__ == "__main__":
682      sys.exit(main())
```